Lab 1.1 - Single Packet Authorization Setup

2026-11-04

Table of contents

1 Introduction **1**
- 1.1 Working With a Partner . . . 2

2 Overview of Commands and Relevant Bash Tips **3**
- 2.1 fwknop . . . 3
- 2.2 iptables . . . 3
- 2.3 ssh . . . 4
- 2.4 scp . . . 4
- 2.5 Bash Tips . . . 4

3 Network Setup **5**

4 Client Installations **6**
- 4.1 fwknop-client . . . 6
- 4.2 nmap . . . 6

5 Server Installations **6**
- 5.1 fwknop-server . . . 6
- 5.2 openssh-server . . . 7

6 Client Setup **7**

7 Server Setup **9**

1 Introduction

This lab contains the setup for a brief tutorial for implementing Single Packet Authorization (SPA) with Firewall Knock Operator (or fwknop for short, pronounced eff - double u - nop).

This lab requires two machines: one designated as the "client" and the other as the "server". See **Working With a Partner** below for how to set this up.

Note that this lab only includes the setup. Lab 1.2 includes the execution of SPA.

1.1 Working With a Partner

Forming groups. Work in groups of three if possible: one server, one client, and one "outsider" machine. The outsider is only needed for a single step in Lab 1.2, where you confirm that the server refuses machines other than the client. If you are in a pair, team up with a neighboring pair for that step, or run a third virtual machine.

Working alone. If you do not have a partner, run both the client and the server as virtual machines on your own computer, connected with a NAT Network. See the **If Bridged Networking Does Not Work** tip in the Network Setup section.

Choosing roles. Decide on roles before installing anything, since each role installs different software:

Role	Installs	Needs
Server	`fwknop-server`, `openssh-server`	`sudo` access (to edit config files and firewall rules)
Client	`fwknop-client`, `nmap`	The server's ip address and a login on the server

If you cannot decide, whoever's machine already has `openssh-server` running (see the openssh-server section) should be the server.

Creating a lab login on the server. The client logs in to the server with `ssh` and `scp`, so the client needs a username and password on the server. Do **not** share the password to your own account. Instead, the server should create a temporary account for the lab:

```
sudo adduser spauser
```

Choose a simple password and give it to your partner. Use `spauser` wherever this lab says `username` for the server. You will delete this account during the clean up at the end of Lab 1.2.

Swapping roles. Each partner should get to be both client and server, but only swap after completing **both** Lab 1.1 and Lab 1.2 together, including the Lab 1.2 clean up. When you swap, the new server installs `fwknop-server` and `openssh-server`, the new client installs `fwknop-client` and `nmap`, and you repeat Lab 1.1 and Lab 1.2 from the beginning. If there is not enough class time, finish the first run through and swap during the next class period.

Keep the same groups. If Lab 1.2 is completed on a different day, sit with the same partner(s) and use the same machines, since the keys and configuration from this lab only exist on those machines.

> Note On Timing
>
> If you finish **Lab: Single Packet Authorization Setup** (this document) with less than 15 minutes left in class, do not begin **Lab: Single Packet Authorization Execution** (part 2). Instead, wait until the next class period to complete part 2.

2 Overview of Commands and Relevant Bash Tips

The following commands will be used in this lab.

2.1 fwknop

The `fwknop` command is used to execute the Firewall Knock Operator software. fwknop is open source software that allows us to easily implement Single Packet Authorization. In short, fwknop will be used on the client to send a SPA packet. On the server, it will be used to verify the packet, add a firewall rule to allow access to the port we want access to for a limited time, then remove the firewall rule after the time expires.

2.2 iptables

The `iptables` command is used to manage the rules that determine what to do with network packets, that is, it is for managing firewall rules.

Common flags used with `iptables` are as follows:

```
-L      lists all the rules in the current tables
-v      displays additional details about each rule (verbose)
-n      displays numeric values where applicable
        ex: for the http port, show 80 instead of http
-A      appends a rule to a specified chain
-p      used to specify a protocol such as tcp for tcp traffic like HTTP and SSH
-j      used to specify an action to take if a packet matches (j for jump)
```

As part of this lab, you will eventually set up a firewall rule that blocks all incoming SSH traffic, so that only fwknop can open the port.

On current Ubuntu, Debian, and Mint releases, `iptables` is a front end for the newer nftables framework. The commands in this lab work the same either way. See the Access Control notes for details.

2.3 ssh

The `ssh` command allows secure access to a remote machine.

2.4 scp

The `scp` command stands for `secure copy`. You can use this command to copy information from one machine to another using SSH. The general format is as follows:

```
scp [options] source destination
```

Common options are as follows:

```
-r     recursively copy subdirectories and files.
-P     specifies the port. The default port is 22.
```

2.5 Bash Tips

For this lab, you will be setting various variables in bash. Recall that in bash, when setting a variable, we do not put spaces on either side of the `=` sign.

Example:

```
x=10    # correct
y = 12  # incorrect
```

3 Network Setup

In order for this lab to work as intended, you may need to modify the settings in your virtualization software and in the network options within your virtual machine. Complete the following to ensure that everything is in working order.

1. If on campus, use `ip addr` to check if your ip address is in the form 138.47.X.X. If so, then you can skip this section. If not, complete the steps that follow.
2. In the virtual machine settings for your virtualization software, ensure that the network is set to use a bridged connection and that the proper connection is selected (most likely Wifi). To do this with VirtualBox, select the machine, open the settings, navigate to `Network` and be sure the adapter is "Attached to" the Bridged Adapter.
3. Within the virtual machine, open the system configuration and navigate to the network settings. Find the settings for IPv4 and make sure DHCP is set to Automatic.
4. At this point you may need to disconnect from the internet and reconnect, or you may need to restart your machine altogether.
5. Once complete, check that your ip address is in the form 138.47.X.X if on campus.

If Bridged Networking Does Not Work

Many campus Wi-Fi networks do not allow bridged virtual machines, because the VM appears with an extra hardware (MAC) address. If your VM does not get a 138.47.X.X address, or the two machines cannot reach each other, run both the client and the server as virtual machines on the same computer and connect them with a private network instead:

- In VirtualBox, open **File > Tools > Network Manager**, select the **NAT Networks** tab, and create a NAT Network. Then, in each VM's `Network` settings, set "Attached to" to **NAT Network** and select the network you created. (A **Host-only Adapter** also works, but those VMs will not have internet access for `apt install`.)
- Each VM will receive an address such as 10.0.2.X instead of 138.47.X.X. Use those addresses everywhere this lab says 138.47.x.x.

This also keeps the lab's port scanning off the campus network.

4 Client Installations

4.1 fwknop-client

1. On the client, update the package lists and search for fwknop in the package directory:

```
sudo apt update
apt search fwknop
```

2. In the output, you should see some of the following (results may vary):

   - fwknop-apparmor-profile
   - fwknop-client
   - fwknop-server

3. Install fwknop-client.

```
sudo apt install fwknop-client
```

4.2 nmap

On the client, if you do not already have nmap installed, install it with the following command.

```
sudo apt install nmap
```

5 Server Installations

5.1 fwknop-server

1. On the server, update the package lists and search for fwknop in the package directory:

```
sudo apt update
apt search fwknop
```

2. In the output, you should see some of the following (results may vary):

   - fwknop-apparmor-profile
   - fwknop-client
   - fwknop-server

3. Install fwknop-server.

```
sudo apt install fwknop-server
```

5.2 openssh-server

On the server, confirm you have the openssh server running on port 22 with the ss command (socket statistics). Using the flags `-a` and `-t` will filter the information down to just the TCP ports. Using the flag `-n` will show you the port number.

```
ss -atn # -n will show the port number (22 for ssh)
ss -at  # no -n will show the label `ssh` in place of the value 22.
```

You should see an output that contains at a minimum the following (results may vary):

```
State       Local Address:Port
LISTEN                *:22
```

or

```
State       Local Address:Port
LISTEN                *:ssh
```

If you don't already have openssh server installed, you will need to install it with the following command.

```
sudo apt install openssh-server
```

After installing it, confirm it is running with the `ss` command shown previously.

6 Client Setup

1. On the client, get your ip address with the `ip` command and note it elsewhere (a text file or a piece of paper) as your client's ip address.

```
# brief, one line per interface
ip -br addr
# full details
ip addr
```

The older `ifconfig` command also works if the `net-tools` package is installed, but it is not installed by default on recent Ubuntu and Mint releases.

2. On the server, get the ip address and note it somewhere (a text file or a piece of paper).

3. On the client, set some variables to represent the `SERVER_IP` and `CLIENT_IP`. Replace the x's in the command shown below with the appropriate values. Recall that bash does not accept spaces before and after the `=` symbol.

```
CLIENT_IP=138.47.x.x
SERVER_IP=138.47.x.x
```

4. On the client, set another variable to represent the server's ssh port.

```
SSH_PORT=22
```

5. On the client, generate the keys and save the settings for this server. This command creates two keys: an encryption key (used to encrypt the SPA packet) and an HMAC key (used to prove the packet was not altered). The keys are not tied to a port. The `--save-rc-stanza` option saves the keys, along with the port to request (`-A`), the client ip (`-a`), and the server ip (`-D`), as a named section ("stanza") in `~/.fwknoprc`. The stanza is named after the server's ip, which is why `fwknop -n $SERVER_IP` works in Lab 1.2. The `\` at the end of the first line continues the command onto the next line, so you can type or paste both lines as shown.

```
fwknop -A tcp/$SSH_PORT -a $CLIENT_IP -D $SERVER_IP \
    --key-gen --use-hmac --save-rc-stanza
```

You should receive an output similar to the following:

```
[+] Wrote Rijndael and HMAC keys to rc file: /home/username/.fwknoprc
```

6. On the client, to confirm the keys were generated, view the contents of the `.fwknoprc` file

```
cat ~/.fwknoprc
```

At a minimum, you should see something similar to the following two lines inside the file:

```
KEY_BASE64          dGhlIGJpZ2dlc3Qgam9rZSBpbiBjb21wdXRlciBzY2llbmNlIGlz
HMAC_KEY_BASE64     eW91ciBwcm9ncmFtcw==
```

7 Server Setup

1. On the client, copy the keys you generated to the server using the `scp` command (secure copy). Note that we are copying the keys to a file called `copied-file.conf` at `/home/username` on the server. This is just to migrate the contents from one machine to another. You'll then take some of the contents of this file and move them elsewhere on the server. Additionally, be sure to replace the word `username` with the name of the user of the server.

```
scp -P $SSH_PORT ~/.fwknoprc username@$SERVER_IP:~/copied-file.conf
```

   When running the above command, you will be prompted for the password of the server.

2. On the server, open the file at `/etc/fwknop/access.conf` and find the section that has the following information:

```
SOURCE              ANY
KEY_BASE64          __CHANGE_ME__
HMAC_KEY_BASE64     __CHANGE_ME__
```

   i. Remove `ANY` and replace it with the ip address for the client machine.

   ii. Replace both instances of `__CHANGE_ME__` by copying and pasting the keys from the `copied-file.conf` file you moved onto the server. If you copied it using the temporary `spauser` account, the file is in `spauser`'s home directory, so view it from your own account with `sudo cat /home/spauser/copied-file.conf`.

   iii. Add a line after SOURCE that is labeled OPEN_PORTS with a value of tcp/22.

   iv. Confirm the file looks similar to the following. The values for SOURCE, KEY_BASE64, and HMAC_KEY_BASE64 will be different from those shown.

```
SOURCE              138.47.163.65
OPEN_PORTS          tcp/22
KEY_BASE64          dGhlIGJpZ2dlc3Qgam9rZSBpbiBjb21wdXRlciBzY2llbmNlIGlz
HMAC_KEY_BASE64     eW91ciBwcm9ncmFtcw==
```

   v. Save and close the file.

3. On the server, remove the copy of the keys you placed in the home directory.

```
sudo rm /home/spauser/copied-file.conf   # if you used the spauser
 ↪  account
rm ~/copied-file.conf                    # if you copied it to your own
 ↪  account
```

4. On the server, find the name of the network interface that has your ip address. fwknopd needs to know which interface to watch for SPA packets.

```
ip -br addr
```

   The interface is the first column of the line containing your ip address. It is typically in the form `enp0s3` (common for VirtualBox) or `eth0`. Values may differ. Note what it is.

5. Complete the following on the server to enable fwknop

   i. Open the file `/etc/default/fwknop-server` with nano or vim, if it exists.

   ii. If it contains `START_DAEMON="no"`, change it to `START_DAEMON="yes"`.

   iii. Save and close the file.

   On systemd-based releases, this setting may be ignored, and step vii (`systemctl enable`) is what actually makes the server start on boot. Changing it does no harm, and it matters on systems that use the older init scripts.

   iv. Open the file `/etc/fwknop/fwknopd.conf` with nano or vim. This is the fwknop daemon config.

   v. Find the line with `PCAP_INTF`. Uncomment it (remove the leading `#`) and replace `eth0` with the interface you found in the previous step. If there is no `PCAP_INTF` line in the file, add one. Note that each line in this file ends with a semicolon. For example:

```
PCAP_INTF                       enp0s3;
```

   If the interface listed here does not exist on your machine, fwknopd will fail to start.

   vi. Save and close the file.

   vii. Enable the fwknop server so that it starts on boot, and restart it so that it picks up the new configuration.

```
sudo systemctl enable fwknop-server
sudo systemctl restart fwknop-server
```

6. On the server, check the status of the fwknop-server to be sure it is running. This will also show some logs at the bottom. The status should show “active (running)”.

```
sudo systemctl status fwknop-server
```

   If it is not running, view the full logs to see why:

```
sudo journalctl -u fwknop-server --no-pager | tail -n 30
```

7. On the server, confirm that `access.conf` is readable only by root, since it now holds your keys.

   ```
   sudo ls -l /etc/fwknop/access.conf
   ```

   The permissions should be `-rw-------` (owner root). If they are not, fix them with `sudo chmod 600 /etc/fwknop/access.conf`.

> **Troubleshooting Tip**
>
> If fwknopd will not start or does not seem to see your SPA packets, you can stop the service and run fwknopd in the foreground with verbose output. It will print every packet it inspects and why it accepts or rejects it. Press `ctrl+c` to stop it, then start the service again.
>
> ```
> sudo systemctl stop fwknop-server
> sudo fwknopd -f -vv -i enp0s3     # replace enp0s3 with your interface
> sudo systemctl start fwknop-server
> ```